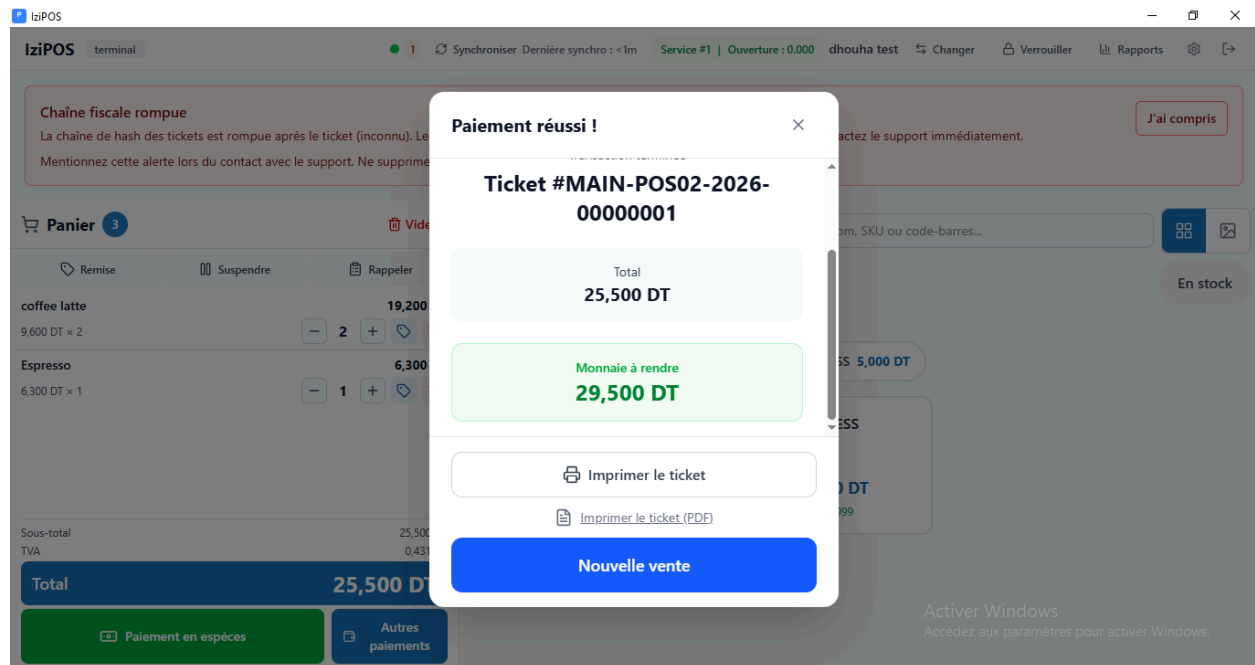


BG 1 :

Clicking on “imprimer le pdf” is not working :



Root-cause analysis for BG 1 :

Root cause (likely):

- apps/pos/src/components/pos/CheckoutSuccessModal.tsx — handlePdfPrint() returns silently when receiptId is falsy (no toast, no log).
- apps/pos/src/pages/HomePage.tsx — the modal reads receiptId from lastReceipt.id; if the offline-first flow returns without populating lastReceipt (e.g. offline and backend-only PDF endpoint), the button is wired to a no-op.
- apps/pos/src/lib/printing.ts — printReceiptAsPdf() hits /api/v1/pos/receipts/{id}/pdf and loads the blob in a hidden iframe. Failures are swallowed (no user feedback).

Check first:

- Open devtools while clicking: is a network request fired at all? If not → receiptId is null.
- Log lastReceipt in paymentStore right after checkout; confirm id is set.
- Confirm backend route apps/api/app/Modules/POS/routes.php exposes the PDF endpoint and returns a valid PDF (not JSON error) for the receipt id.

BG 2 :

Root-cause analysis for BG 2 :

Root cause (likely):

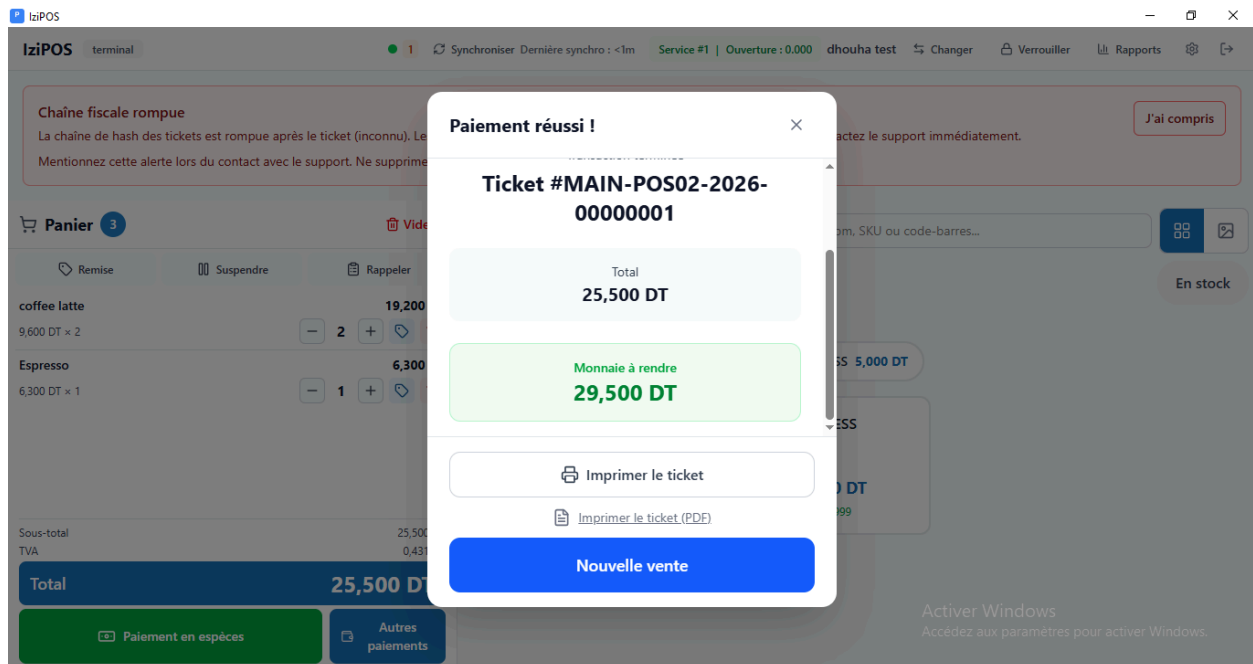
- apps/pos/src/lib/offline/receiptService.ts generateReceiptNumber() uses terminal_state.hash_sequence (read via terminalStateRepository). If hash_sequence is reset on every app launch (or the terminal_state row is recreated at bootstrap), every sync produces sequence=1.

- apps/pos/src/stores/paymentStore.ts — the receipt number displayed comes from offline generation, not from the backend response, so a backend counter wouldn't fix this if client-side sequence isn't persisted.

Check first:

- Query the local SQLite terminal_state table after 2–3 sales: is hash_sequence growing? If stuck at 0/1 → persistence is the bug.
- Search advanceHashChain() in terminalStateRepository.ts — verify it commits the row (not just an in-memory mutation).
- Check that terminal bootstrap does not call upsert with hash_sequence = 0 on every boot.

Ticket number is always 1 :



BG3 :

When calling back a suspended receipt i got a 0 as total :

Root-cause analysis for BG 3 :

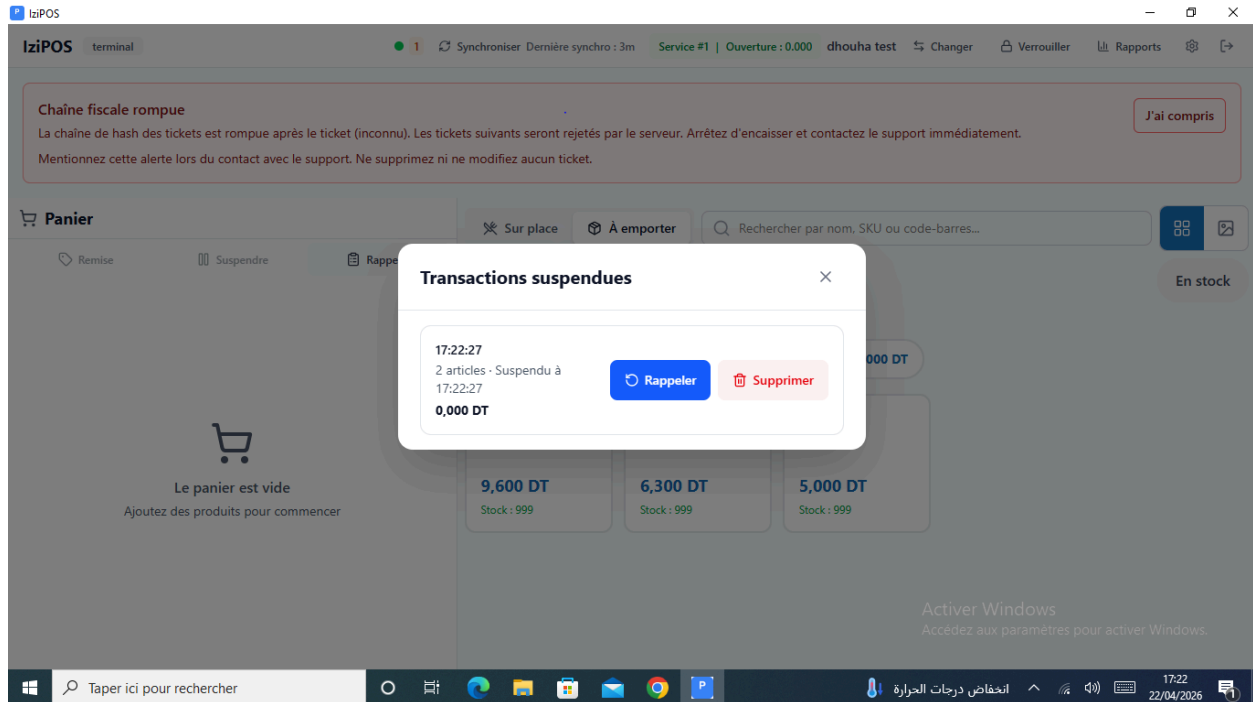
Root cause (likely):

- apps/pos/src/stores/holdStore.ts — saves items and transactionDiscount, but HomePage.handleRecall() restores items one-by-one via useCartStore.setState without triggering the recomputation path (addItem/setItems).
- cartStore.total() is a selector (derives from subtotal - discount), so totals should recompute — unless items are pushed without normalizing line_total/price fields (the stored snapshot may have shape X, cartStore expects shape Y).

Check first:

- Log cart state right after recall: are items present but line_total / unit_price fields missing or zero?

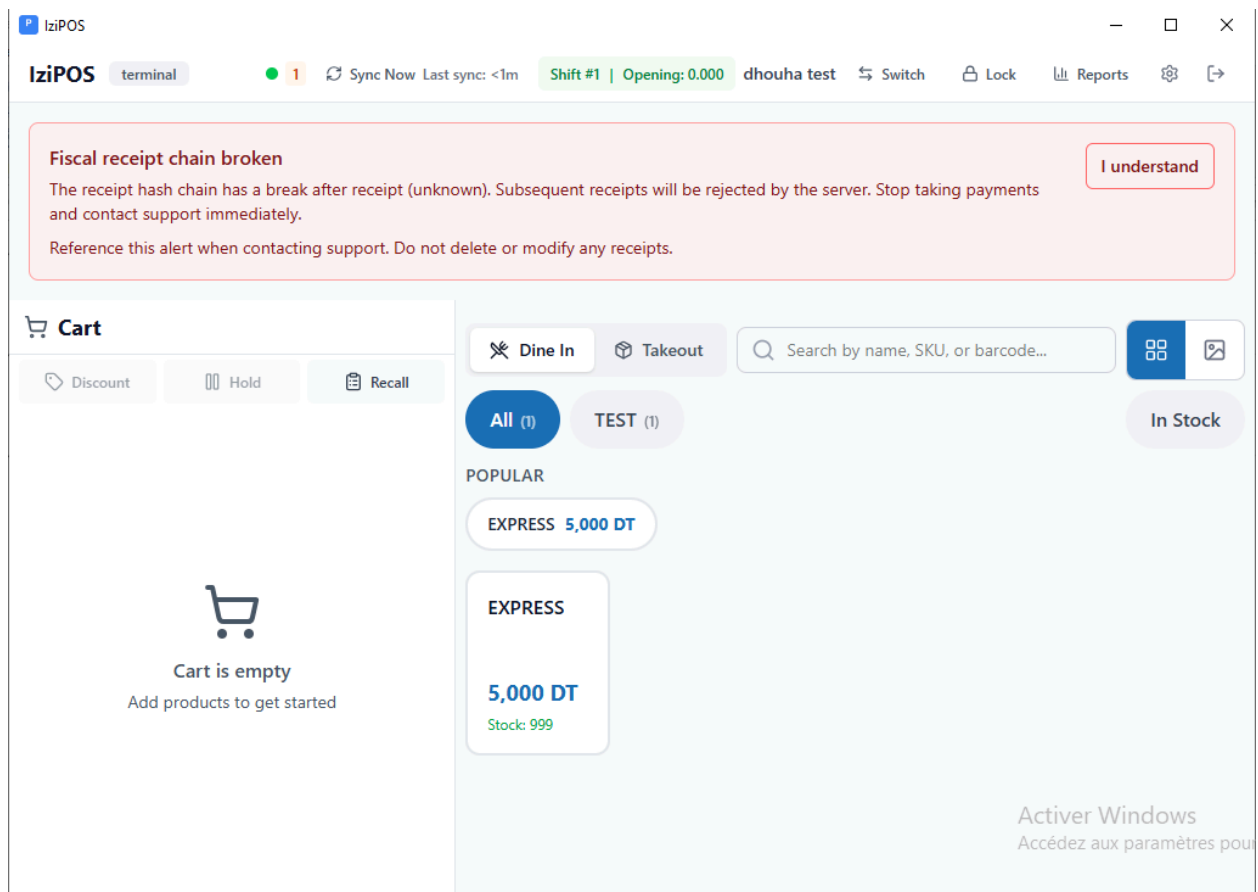
- Compare the item shape saved in `holdStore.holdTransaction()` vs the shape `cartStore.addItem()` produces.
- Replace the raw `setState` loop in `handleRecall` with a proper `replaceCart(items)` action in `cartStore` that re-derives line totals.



BG 4 :

Syncronization is done only when clicking on sync , and in most of times is not done just after clicking multiple times on sync + when clicking multiple times on it i got the fiscal chain hash

broken :



Root-cause analysis for BG 4 :

Root cause (likely):

- apps/pos/src/lib/sync/syncScheduler.ts — auto-sync is interval-based (BASE_INTERVAL_MS = 60_000). No event-driven trigger when a receipt is posted offline, so queue sits until the next tick or a manual click.
- apps/pos/src/stores/syncStore.ts triggerSync() guards only against concurrent runs (isSyncing flag) — rapid successive clicks can race past it before the flag is set.
- apps/api/app/Modules/POS/Application/Services/ReceiptSyncService.php — chain verification compares client previous_hash to terminal.last_hash. On a successful push, last_hash advances. A retry with the old previous_hash fails with "chain broken". The idempotency_key dedupe exists for receipt-id duplicates but does not rescue a client that has not refreshed its local chain head.

Check first:

- Add an event-driven trigger: when offlineReceiptsRepository inserts a pending receipt, enqueue a sync immediately (debounced).
- Make the Sync button disabled while isSyncing is true, AND add a 500ms debounce on the click handler.
- After a successful push response, update the client's local last_hash from the server response before allowing any further push. If the client does not know the advanced hash, the queue is poisoned.

Fiscal receipt chain broken

The receipt hash chain has a break after receipt (unknown). Subsequent receipts will be rejected by the server. Stop taking payments and contact support immediately.

Reference this alert when contacting support. Do not delete or modify any receipts.

[I understand](#) **Cart** Discount Hold Recall Dine In Takeout Search by name, SKU, or barcode... 

All (1)

TEST (1)

In Stock

POPULAR

EXPRESS 5,000 DT

EXPRESS

5,000 DT

Stock: 999

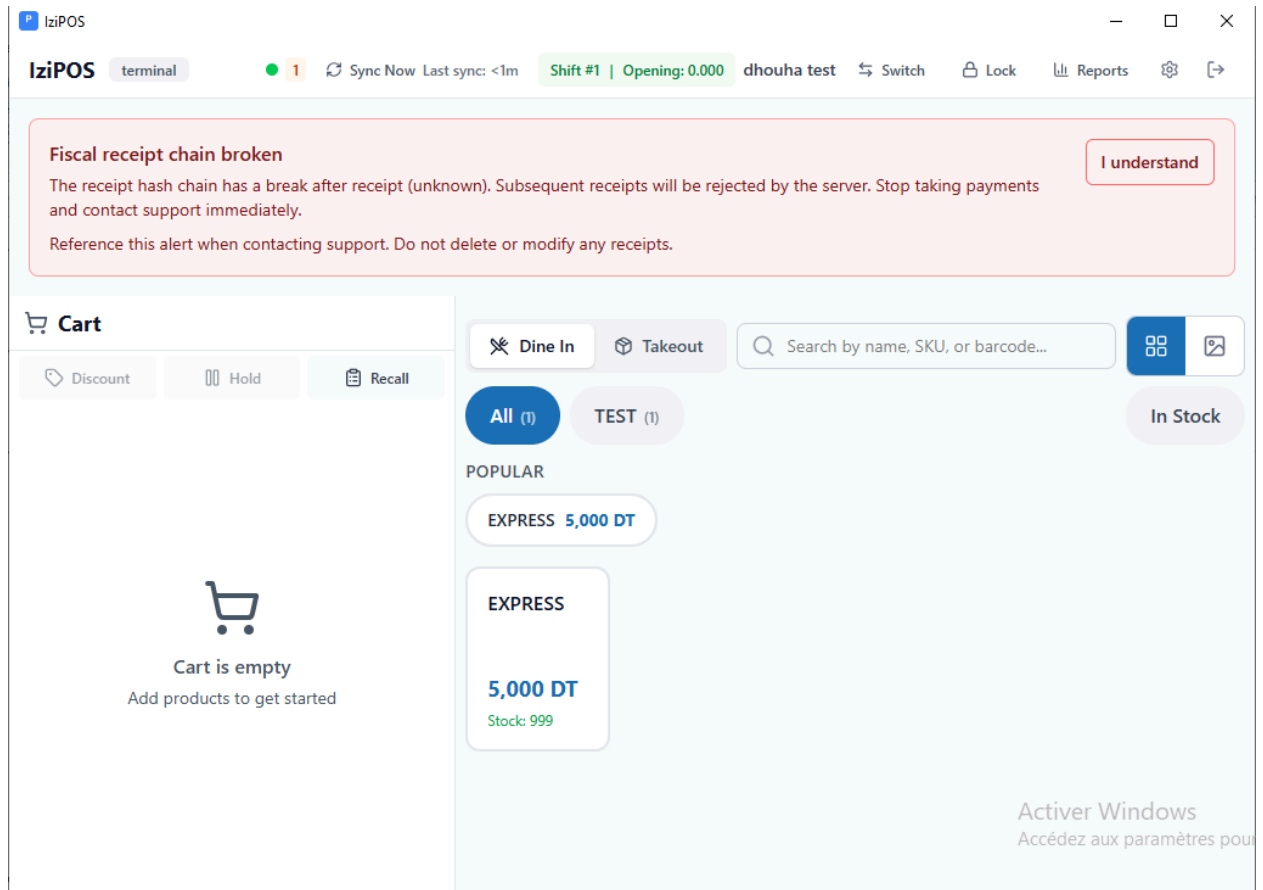


Cart is empty

Add products to get started

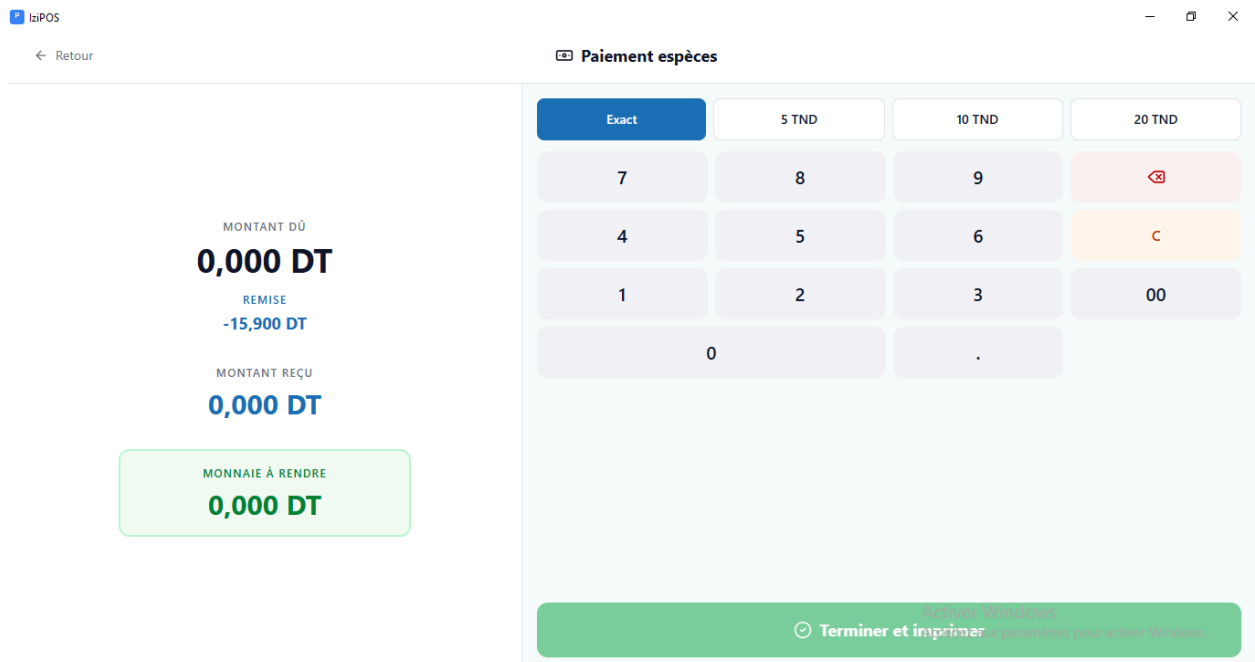
Activer Windows

Accédez aux paramètres pour



BG 5 :

Total is not calculated correctly for suspended and recalled receipt + change due :



BG6 :

Change due on tickets (printed) is always 0 :

test dhouha

Tax ID:

Receipt: MAIN-POS02-2026-00000001
Date: 2026-04-22 10:33:18
Terminal: POS02
Operator: dhouha test

Item	Qty	Amount
------	-----	--------

EXPRESS		
1 x 5.000		TND5.000

Subtotal:	TND5.000
Discount:	-TND0.000
Tax:	TND0.798
TOTAL:	TND5.000

Payments:

Esp	TND5.000
Change Due:	TND0.000

Hash: 0c510b5d1781a0fd...43c02f1d00f142c5



Thank you for your purchase!

Root-cause analysis for BG 5 :

Root cause (likely):

- Same as BG3 — recalled cart totals stay at 0 because line items aren't normalized on restore, so `cartStore.total() = 0`.
- Change due derives from `tenderedAmount - total`. If total is 0 on the recalled cart, `change due = tenderedAmount` (which looks wrong to the user and gets printed as 0 later — see BG6).

Check first:

- Fix BG3 first (replaceCart action) and re-test BG5.
- Verify CashPaymentScreen receives the recomputed total as a prop after recall.

Root-cause analysis for BG 6 :

Root cause (likely):

- `changeDue` is client-only state in `paymentStore`. It is passed into `CheckoutSuccessModal` for display, but there is no evidence it is persisted to the Receipt domain model on the backend.
- `apps/api/app/Modules/POS/Application/Services/ReceiptPdfService.php` — template renders what's on the Receipt aggregate. If the aggregate has no `change_due` column/field, the PDF prints 0.

Check first:

- Grep the backend migrations for `change_due / monnaie_rendue` on the `pos_receipts` table. If the column does not exist, add it (or compute it at print time as `tendered_total - total`).
- In the offline→server sync payload, include `change_due` so the server persists it.
- In `ReceiptPdfService.prepareData()`, compute `change_due` from payments if not stored (sum of cash tendered minus receipt total).

BG 7 :

Deleted items if menus still appear (today they disappear), synchronisation is not done simultaneously (even when clicking many times on sync)

 Cart Discount Hold Recall Dine In Takeout Search by name, SKU, or barcode... 

All (1)

TEST (1)

In Stock

POPULAR

EXPRESS 5,000 DT

EXPRESS

5,000 DT

Stock: 999

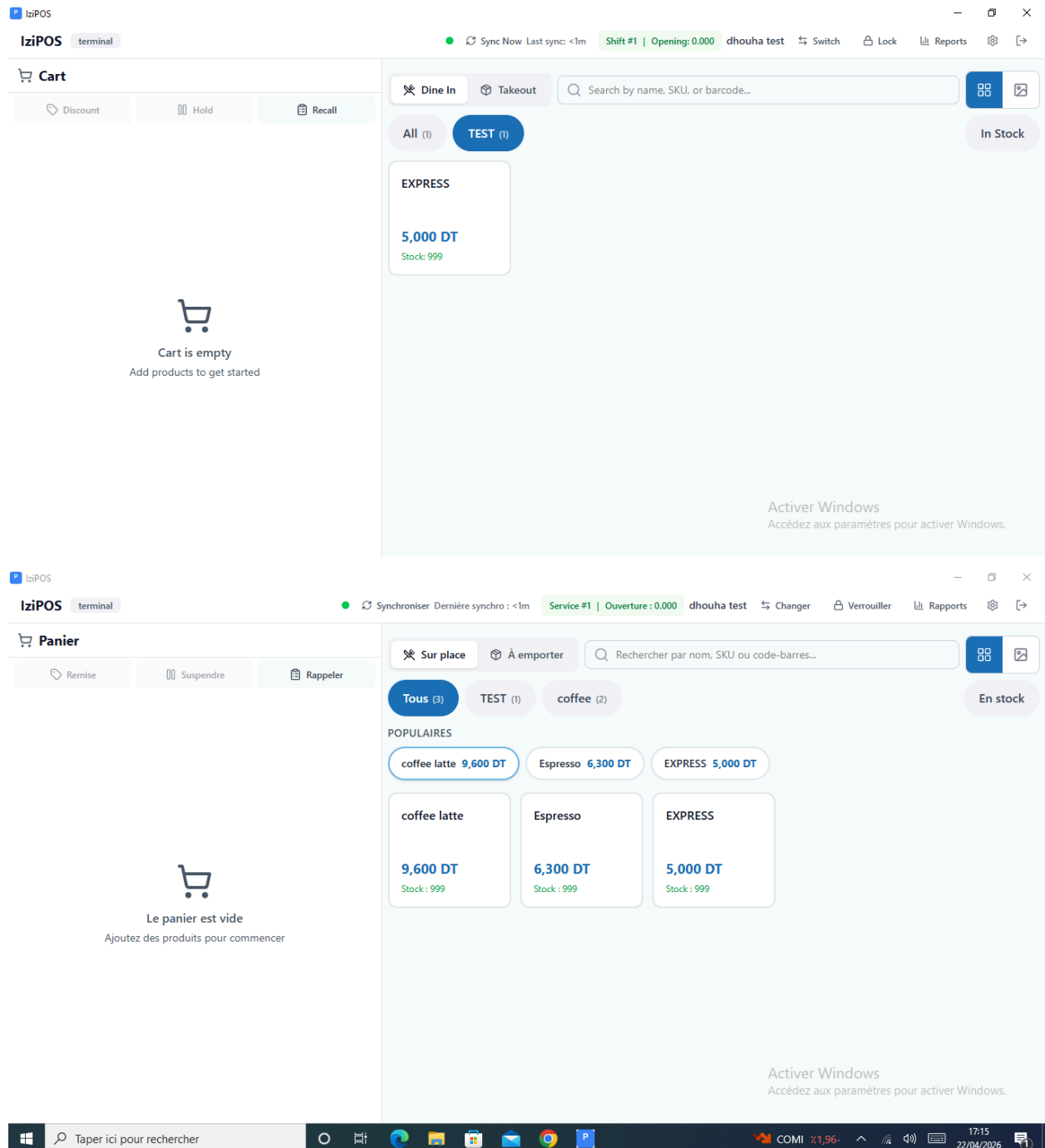


Cart is empty

Add products to get started

Activer Windows

Accédez aux paramètres pour activer Windows.



Root-cause analysis for BG 7 :

Root cause (likely):

- apps/api/app/Modules/Menu/Application/DTOs/ActiveMenuData.php — the active-menu query does not call withoutTrashed() on items/composites. MenuCategory.compositeItems() / .products() are belongsToMany with no deleted_at filter, so soft-deleted items still surface in the payload.

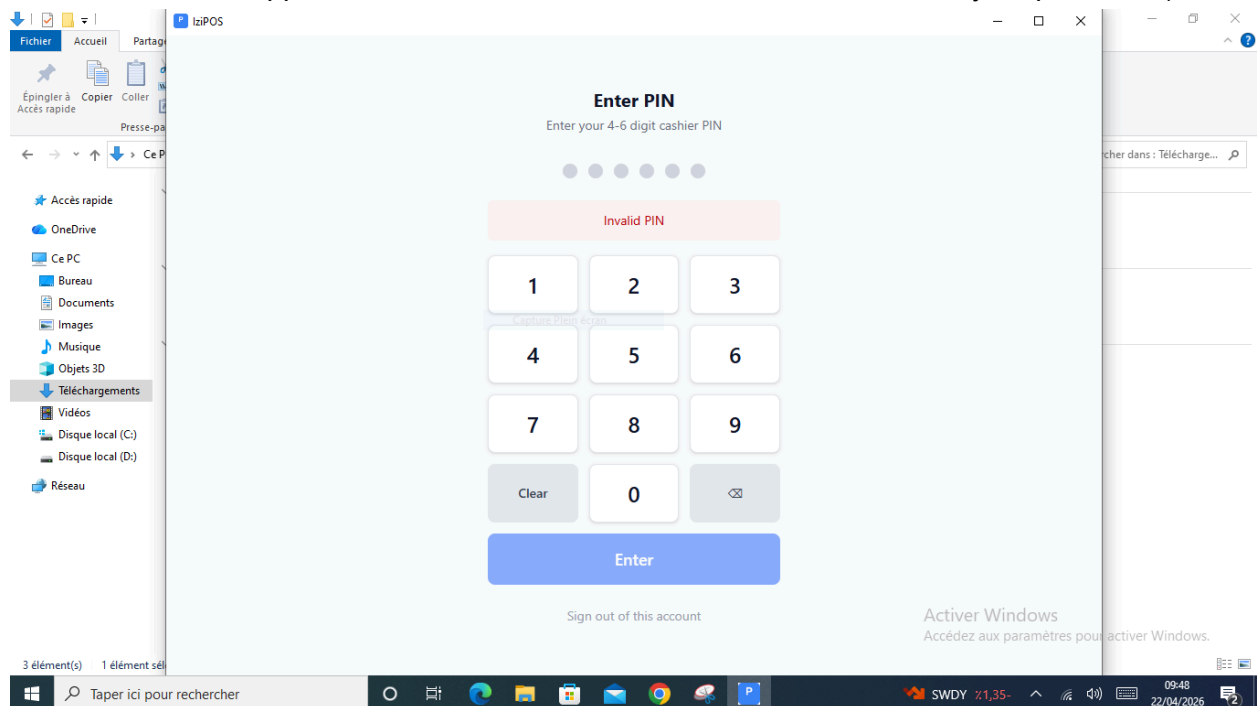
- apps/pos/src/lib/db/repositories/productRepository.ts — upsertProducts() is append-only (INSERT OR UPDATE). It never DELETES rows absent from the sync response, so once a product reaches the local SQLite cache, nothing removes it. No tombstone tracking.
- The "sync is not simultaneous" part is the same as BG4 — interval-based only, no push-on-change.

Check first:

- Add ->whereNull('deleted_at') (or ->withoutTrashed()) on the active-menu joins.
- Either (a) switch the sync to full snapshot + DELETE rows not present, or (b) send explicit deletion markers and have the client repository apply DELETES.
- Verify that soft-deleting a menu item in the Admin UI clears it from /api/v1/pos/menu/active immediately.

BG 8 :

Sometimes , the pin have to be inserted 2 times so user can login to the terminal (this is a random behavior happens when online connected and offline and not always reproduced) :



Root-cause analysis for BG 8 :

Root cause (likely):

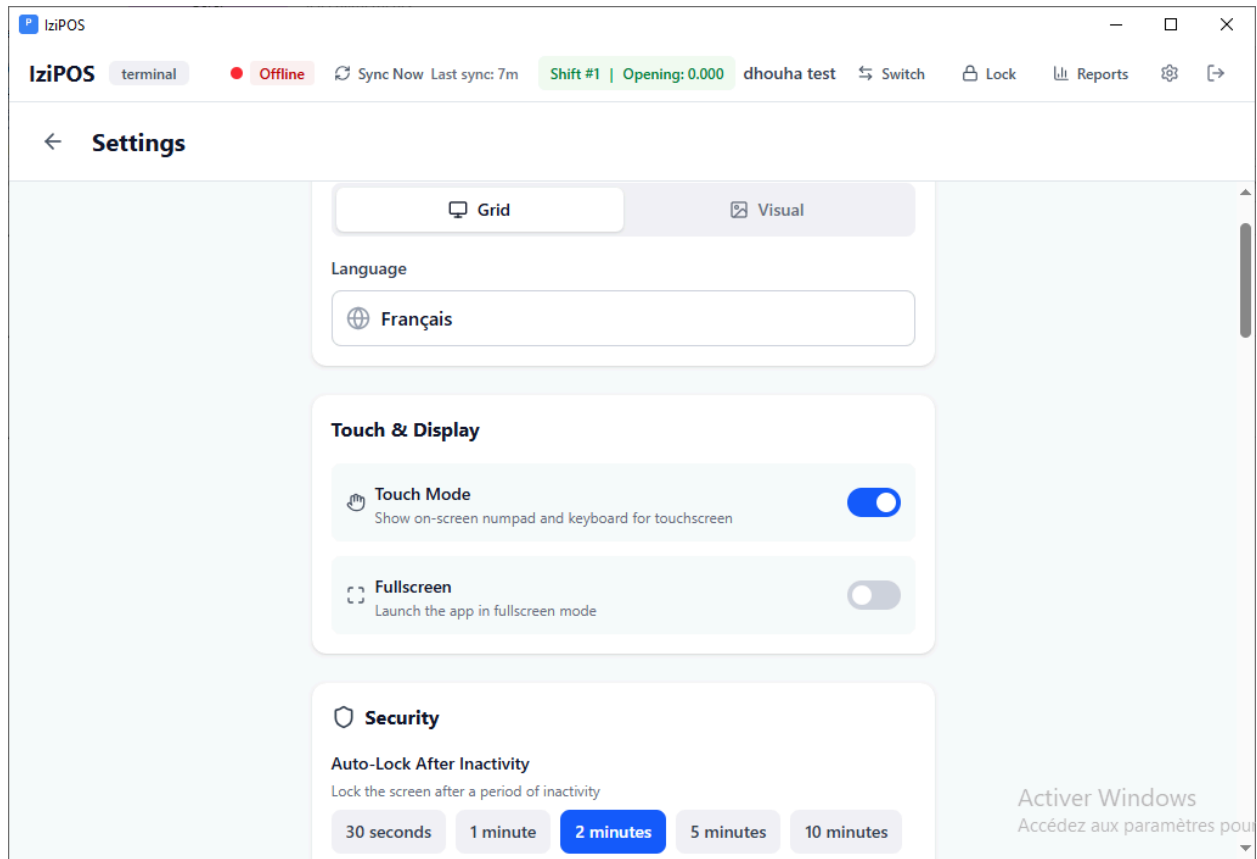
- apps/pos/src/stores/operatorStore.ts verifyPin() — tries the API first, falls back to offline bcrypt only on throw. On a flaky first call (timeout, DNS, 5xx), the first attempt fails-visibly and the user retries; the second attempt is served by the cached bcrypt path. Looks like "PIN needed twice".
- apps/pos/src/pages/PinEntryPage.tsx handleSubmit — clears pin on error without a submission lock; a second tap during in-flight re-submits.
- Also possible: Sanctum CSRF cookie not fetched before the first verify call — second call succeeds once the cookie is set.

Check first:

- Log both paths (online/offline) in verifyPin. Confirm which branch wins the "second try".
- Check apiClient bootstrap: is /sanctum/csrf-cookie hit before the first auth call?
- Add an in-flight guard on handleSubmit and disable the PinPad while verifying.

BG 9:

Full screen is sometimes not working the right way (it failed first , second , third time and then always it is done the right way)



Root-cause analysis for BG 9 :Root cause (likely):

- apps/pos/src/lib/fullscreen.ts applyFullscreen() chains four Tauri window APIs (setDecorations, setFullscreen, setSkipTaskbar, setAlwaysOnTop). Any individual failure is swallowed into a console.error and the chain halts.
- apps/pos/src/App.tsx — the effect that calls applyFullscreen() depends on settingsStore.fullscreen. Once it's true, the effect does not re-run even if the actual window did not go fullscreen (state/reality desync).
- On the first click after cold start, the Tauri window may not be fully initialized; the API returns but the OS hasn't applied it yet.

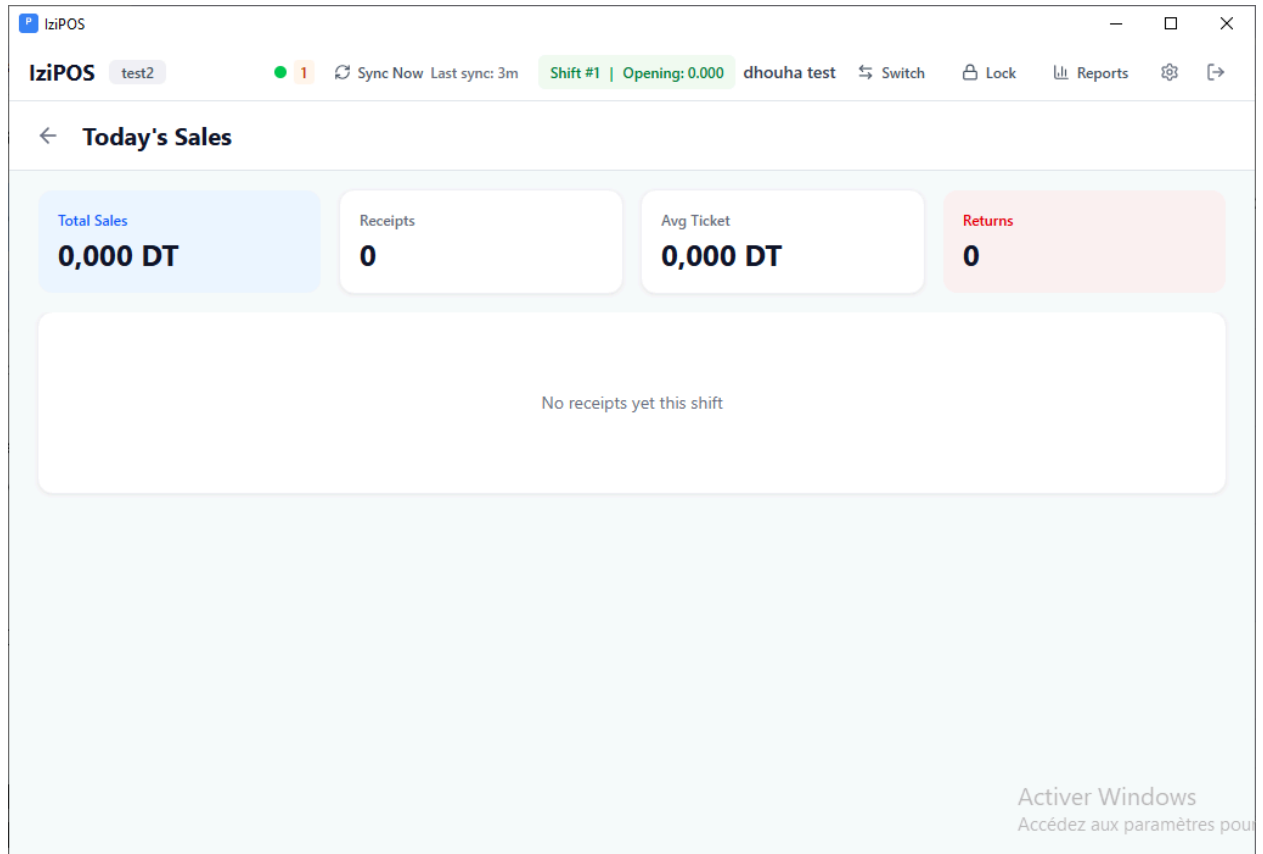
Check first:

- Await each Tauri call individually with try/catch; on failure, schedule a single retry after ~250ms.
- After applying, read back window.isFullscreen() and compare to the desired state; if mismatch, retry.

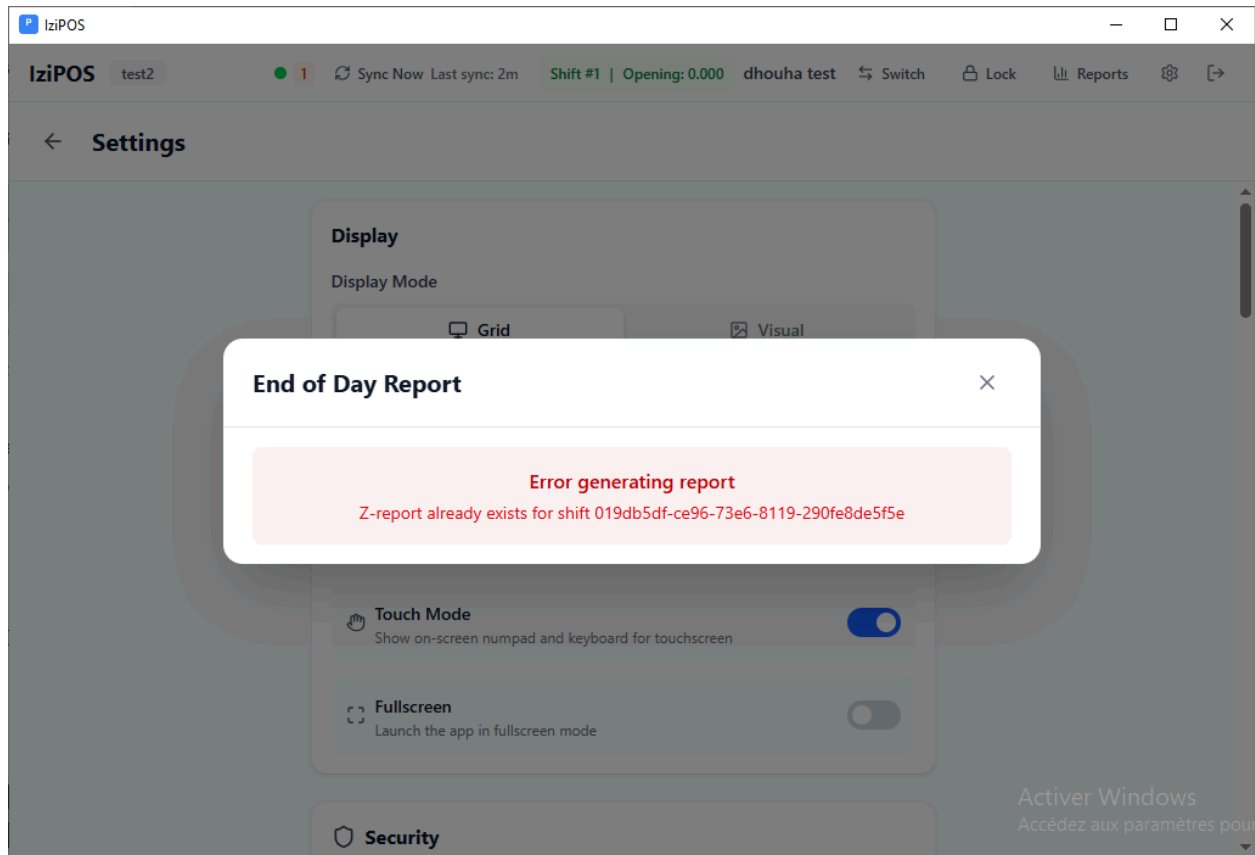
- Add a manual "Force fullscreen" fallback in Settings so users aren't blocked while this is flaky.

BG 10:

History is always empty :



Rapport Z : never generated :



Root-cause analysis for BG 10 :

Root cause (likely):

- apps/pos/src/components/pos/TodaySalesPanel.tsx — loadReceipts() derives shiftId from shift?.id; if no shift is open (or if terminalStore hasn't hydrated shift from backend), fetchShiftReceipts is never called and the list stays empty silently.
- apps/pos/src/api/reportApi.ts fetchShiftReceipts — on API error falls back to fetchLocalShiftReceipts, which returns [] if the local SQLite has nothing for that shift window; no error surfaced to the UI.
- apps/api/app/Modules/POS/Presentation/Controllers/ShiftController.php close() — marks the shift CLOSED but does NOT invoke Z report generation. Z is only produced via the explicit Reports → Rapport Z action. If the cashier never opens that menu, no Z is ever created.

Check first:

- In devtools, log shift?.id when mounting TodaySalesPanel — is it undefined?
- Hit /api/v1/pos/shifts/{shiftId}/receipts manually with a known shift id; confirm it returns rows.
- Decision to make: should close-shift auto-generate the Z report, or should the UI force the Z-report modal before allowing close? Currently neither happens.